

LAPORAN MODUL 11

PRAKTIKUM

PARADIGMA PEMGROGRAMAN



Disusun oleh :

Nama : Irvan Cahya Nugraha
NIM : 245410034
Kelas : Informatika 1

PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025

MODUL 11

POLIFORSISME

A. TUJUAN PRAKTIKUM

1. Dapat membuat aplikasi yang mengimplementasi compile-time (early binding) polimorphisme, dapat membuat aplikasi yang mengimplementasi runtime (late binding) polimorphisme

B. DASAR TEORI

Polimorfisme adalah kemampuan sebuah variabel reference untuk merubah behavior sesuai dengan apa yang dipunyai object. Polimorfisme membuat objek-objek yang berasal dari subclass yang berbeda, diperlakukan sebagai objek-objek dari satu superclass. Hal ini terjadi ketika memilih method yang membedakan method berdasarkan signature method (saat compile) atau berdasarkan reference object (saat runtime). Ada dua bentuk polimorfisme

1. Overloading (compile-time (early binding) polimorphisme)

Overloading adalah salah satu cara penerapan dalam konsep polimorfisme. Overload merupakan pendefinisian ulang suatu metode dalam class yang sama. Syarat overload yaitu metode dan tipe parameter harus berbeda dalam kelas yang sama.

2. Overriding (runtime (late binding) polimorphisme)

Override merupakan pendefinisian ulang suatu metode oleh subclass. Syarat Override yaitu metode, return type, dan parameter harus sama. Jika tidak sama maka bukan dianggap sebagai override tetapi metode yang baru pada subclass. Dikatakan **late-binding (run time) polymorphism** karena saat compile komputer tidak tahu method mana yang harus dipanggil, dan baru akan diketahui saat run-time. Run-time polymorphism dapat diterapkan melalui **overridden method**. Run time polymorphism juga disebut dengan istilah **dynamic binding**. Anggaplah subclass meng-override method tertentu pada superclass. Andai kita cipta objek dari subclass dan memasukkannya dalam superclass. Walau objek subclass telah dimasukkan pada superclass, sewaktu objek tersebut memanggil method yang dioverride, ia tetap memanggil method yang versi subclassnya, bukan versi superclass.

C. PEMBAHASAN LISTING

PRAKTIK

Praktik 1

Pembahasan :

LATIHAN

D. PEMBAHASAN TUGAS

```

import java.util.Scanner;

// Supercelas
abstract class Karyawan {
    String nama;

    public Karyawan(String nama) {
        this.nama = nama;
    }

    public abstract double hitungGaj ();

    public void tampilkanDat () {
        System.out.println("\n--- DATA GAJI KARYAWAN ---- ");
        System.out.println("Nama Karyawan :   + nama);
        System.out.println("Total Gaji   :   + hitungGaj ();
    }
}

// Subclass: Karyawan Teta
class karTap extends Karyawan {
    double gapok;

    public karTap(String nama, double gapok) {
        super(nama);
        this.gapok = gapok;
    }

    @Override
    public double hitungGaj () {
        double tunjangan = 0.2 * gapok;
        return gapok + tunjangan;
    }
}

// Subclass: Karyawan Kontra
class Kkontrak extends Karyawan {
    int jamKerja;
    double upahPerJa ;

    public Kkontrak(String nama, int jamKerja, double upahPerJa ) {
        super(nama);
        this.jamKerja = jamKerja;
        this.upahPerJa = upahPerJa ;
    }

    @Override
    public double hitungGaj () {
        return jamKerja * upahPerJa ;
    }
}

// Subclass: Karyawan Magan
class KMagang extends Karyawan {
    double uangSaku;

    public KMagang(String nama, double uangSaku) {
        super(nama);
        this.uangSaku = uangSaku;
    }

    @Override
    public double hitungGaj () {
        return uangSaku;
    }
}

public class main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Karyawan karyawan;

        System.out.println("---- PILIH JENIS KARYAWAN ---- ");
        System.out.println("1. Karyawan Teta ");
        System.out.println("2. Karyawan Kontra ");
        System.out.println("3. Karyawan Magan ");
        System.out.print("P&Iltihan: ");
        int pilihan = input.nextInt();
        input.nextLine();

        System.out.print("Masukkan Nama Karyawan: ");
        String nama = input.nextLine();

        if (pilihan == 1) {
            System.out.print("Masukkan Gaji Pokok: ");
            double gaji = input.nextDouble ();
            karyawan = new karTap(nama, gaji);
        } else if (pilihan == 2) {
            System.out.print("Masukkan Jam Kerja: ");
            int jam = input.nextInt();
            System.out.print("Masukkan Upah per Jam: ");
            double upah = input.nextDouble ();
            karyawan = new Kkontrak(nama, jam, upah);
        } else if (pilihan == 3) {
            System.out.print("Masukkan Uang Saku: ");
            double saku = input.nextDouble ();
            karyawan = new KMagang(nama, saku);
        } else {
            System.out.println("Pilihan tidak vali ");
            return;
        }

        // Polimorfisme: Memanggil method melalui referensi superclas
        karyawan.tampilkanDat ();

        System.out.println("Press any key to continue . . ");
    }
}

```

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE COMMENTS
● PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\11> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\3ed897aae96f7c5e707e6d842409ed39\redhat.java\jdt_ws\11_6ab36d74\bin' 'main'
---- PILIH JENIS KARYAWAN ----
1. Karyawan Tetap
2. Karyawan Kontrak
3. Karyawan Magang
Pilihan: 1
Masukkan Nama Karyawan: regi
Masukkan Gaji Pokok: 1000000

---- DATA GAJI KARYAWAN ----
Nama Karyawan : regi
Total Gaji : 1200000.0
Press any key to continue . . .
● PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\11> d; cd 'd:\AREAS\Semester3\Praktikum Paradigma Pemrograman\11'; & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\3ed897aae96f7c5e707e6d842409ed39\redhat.java\jdt_ws\11_6ab36d74\bin' 'main'
---- PILIH JENIS KARYAWAN ----
1. Karyawan Tetap
2. Karyawan Kontrak
3. Karyawan Magang
Pilihan: 2
Masukkan Nama Karyawan: nikita
Masukkan Jam Kerja: 8
Masukkan Upah per Jam: 15000

---- DATA GAJI KARYAWAN ----
Nama Karyawan : nikita
Total Gaji : 120000.0
Press any key to continue . . .
● PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\11> d; cd 'd:\AREAS\Semester3\Praktikum Paradigma Pemrograman\11'; & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\3ed897aae96f7c5e707e6d842409ed39\redhat.java\jdt_ws\11_6ab36d74\bin' 'main'
---- PILIH JENIS KARYAWAN ----
1. Karyawan Tetap
2. Karyawan Kontrak
3. Karyawan Magang
Pilihan: 3
Masukkan Nama Karyawan: gojo
Masukkan Uang Saku: 2000000

---- DATA GAJI KARYAWAN ----
Nama Karyawan : gojo
Total Gaji : 2000000.0
Press any key to continue . . .
○ PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\11> |
```

Pembahasan : Program di atas menerapkan konsep abstract class dan polymorphism. Class Karyawan berperan sebagai superclass yang menyimpan atribut nama dan memiliki method abstract hitungGaji(), sehingga setiap class turunan wajib mengimplementasikan cara perhitungan gajinya masing-masing. Class karTap menghitung gaji berdasarkan gaji pokok ditambah tunjangan, class Kkontrak menghitung gaji dari hasil perkalian jam kerja dan upah per jam, sedangkan class KMagang hanya mengembalikan nilai uang saku. Pada bagian main, objek karyawan dibuat sesuai pilihan user namun disimpan dalam referensi Karyawan. Ketika method tampilkanData() dipanggil, Java secara otomatis menjalankan method hitungGaji() milik class turunan yang sesuai, sehingga konsep polymorphism benar-benar diterapkan.

E. KESIMPULAN

Program ini membuktikan bahwa abstract class digunakan untuk mendefinisikan kerangka umum, sedangkan detail implementasi diserahkan ke class turunan. Dengan menerapkan polimorfisme, perhitungan gaji untuk berbagai jenis karyawan dapat dilakukan melalui satu referensi yang sama tanpa perlu mengetahui tipe objek secara langsung. Pendekatan ini membuat program lebih fleksibel, terstruktur, dan mudah dikembangkan.

F. LAMPIRAN